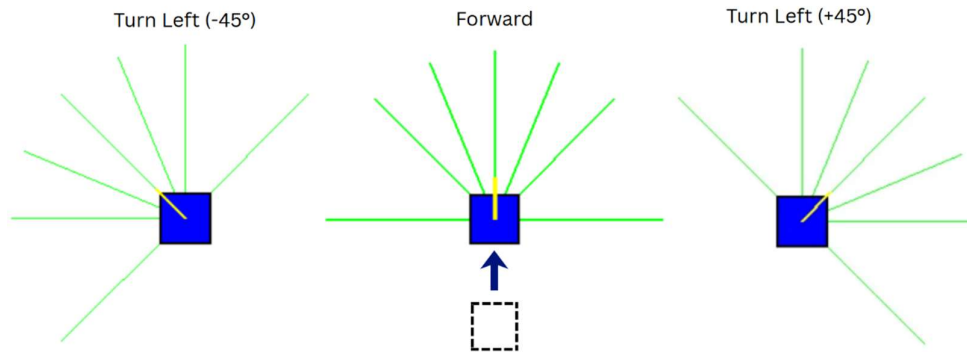


THE AGENT

A) Actions

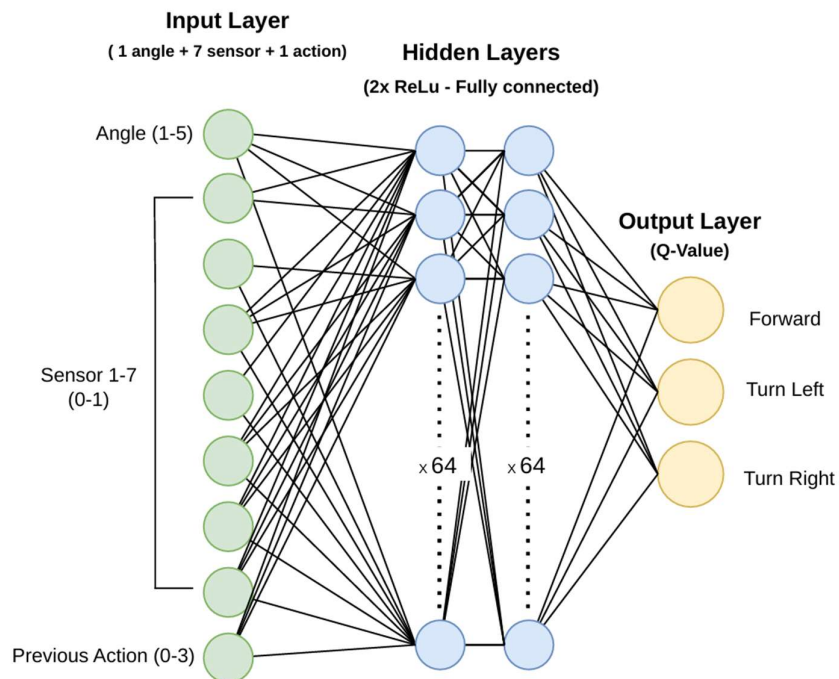
Note: Agent's angle limit is 90 degrees.



B) Rewards

Reward/Penalty	Value	Keterangan
Clear Forward (CF)	2	Apabila sensor tengah kosong dan action adalah "Forward"
Success	50	Apabila agent mencapai <i>finish</i>
Turn Limit (TL)	-1	Apabila agent mencoba belok melebihi limit
Turn Repeat (TR)	-1	Apabila agent melakukan pola <i>turning</i> berulang (<i>ex: left -> right</i>)
Timeout	-25	Apabila <i>step</i> melebihi batas timeout (150 steps)
Collision	-25	Apabila agent melakukan tabrakan

C) Neural Network (Input & Output)



D) Representations Input Output

Berikut Tabel representasi seluruh **9 Neuron Inputs**.

Neuron Input	Value	Normalized
Angle	$[-90^\circ, -45^\circ, 0^\circ, 45^\circ, 90^\circ]$	$[1, 2, 3, 4, 5]$
Sensors 1–7	1 (No Detect) – 0 (Detect)	$1.00 - 0.00$
Previous Action	[No Action, Forward, Turn Left, Turn Right]	$[0, 1, 2, 3]$

Berikut Tabel representasi seluruh **3 Neuron Outputs**.

Q-Value	Action	Value
Q1	Forward	0
Q2	Turn Left	1
Q3	Turn Right	2

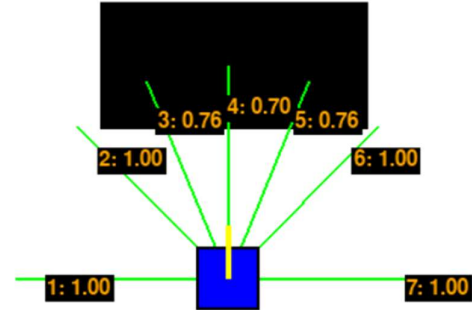
Feedforward:

Input > Output > Action

(Code A: Feed Forward)

A) Agent Inputs

Input	Value	Normalized
Angle	0°	3.00
Sensor 1	1.00	1.00
Sensor 2	1.00	1.00
Sensor 3	0.76	0.76
Sensor 4	0.70	0.70
Sensor 5	0.76	0.76
Sensor 6	1.00	1.00
Sensor 7	1.00	1.00
Previous Action	Forward (permisalan)	1.00



B). Network Architecture

Model Structure:

9 → 64 → 64 → 3

Total weights:

Layer 1-2 : $9 \times 64 = 576$

Layer 2-3 : $64 \times 64 = 4096$

Layer 3-4 : $64 \times 3 = 192$

$576 + 4096 + 192 = 4864 \text{ weights}$

Example weight matrix:

Layer 1: $[w_{1,1} \ w_{1,2} \ \dots \ w_{1,9}] \rightarrow h_{1,1}$ $[w_{2,1} \ w_{2,2} \ \dots \ w_{2,9}] \rightarrow h_{1,2}$... $[w_{64,1} \ w_{64,2} \ \dots \ w_{64,9}] \rightarrow h_{1,64}$ Layer 2: $[w_{1,1} \ w_{1,2} \ \dots \ w_{1,64}] \rightarrow h_{2,1}$ $[w_{2,1} \ w_{2,2} \ \dots \ w_{2,64}] \rightarrow h_{2,2}$... $[w_{64,1} \ w_{64,2} \ \dots \ w_{64,64}] \rightarrow h_{2,64}$ Output Layer: $[w_{1,1} \ w_{1,2} \ \dots \ w_{1,64}] \rightarrow Q0 \text{ Forward}$ $[w_{2,1} \ w_{2,2} \ \dots \ w_{2,64}] \rightarrow Q1 \text{ Turn Left}$ $[w_{3,1} \ w_{3,2} \ \dots \ w_{3,64}] \rightarrow Q2 \text{ Turn Right}$
--

C) Feed Forward

Formulation:

$$h_{1,j} = \text{ReLU} \left(b_{1,j} + \sum_{i=1}^9 W_{1(j,i)} x_i \right)$$
$$h_{2,j} = \text{ReLU} \left(b_{2,j} + \sum_{i=1}^{64} W_{2(j,i)} h_{1,i} \right)$$
$$Q_j = b_{3,j} + \sum_{i=1}^{64} W_{3(j,i)} h_{2,i}$$

Keterangan:

- $j = 1, \dots, 64$ for Hidden Layer 1
- $j = 1, \dots, 64$ for Hidden Layer 2
- $j = 0, 1, 2$ for the output layer (*Forward, Turn Left, Turn Right*)

Ringkasnya:

- **9 operasi perkalian** dilakukan pada setiap neuron di **hidden layer pertama**.
- **64 operasi perkalian** dilakukan pada setiap neuron **di hidden layer kedua**.
- **64 operasi perkalian** dilakukan untuk menghitung setiap **Q-value** pada **output layer**.
- **ReLU**, jika nilai **input bernilai negatif**, maka keluarannya menjadi 0.
- **ReLU**, jika nilai **input bernilai positif**, maka keluarannya tetap sama dengan nilai masukan tersebut.

Contoh ReLU:

Condition	Values
Before ReLU (z)	$[-2, 0, 0, 5, 3, 1, -0, 7]$
After ReLU (h)	$[0, 0, 5, 3, 1, 0]$

D) Aktivasi Hidden Layer Terakhir

Dari 64 neuron pada hidden layer ke-2, ada 33 neuron berikut yang aktif atau bernilai > 0 akibat fungsi ReLU:

h2[0] = 5.79128361
h2[1] = 3.19833732
h2[3] = 2.18439412
h2[4] = 2.30266738
h2[5] = 2.74005818
h2[6] = 2.44832206
h2[8] = 5.34804916
h2[9] = 0.09281521
h2[10] = 1.13427794
h2[12] = 1.32081521
h2[13] = 0.45096001
h2[16] = 1.01145613
... and 19 more active neurons

E) Perhitungan Q-Value (Final Layer ke Output)

Q-value pada output layer dihitung menggunakan persamaan:

$$Q_j = b_{3,j} + \sum_{i=1}^{64} W_{3(j,i)} h_{2,i}$$

- Q_j : Nilai Q-value untuk aksi ke- j , yang menunjukkan estimasi nilai *action*.
- $b_{3,j}$: Bias pada neuron ke- j di output layer. Bias merupakan parameter yang dipelajari selama proses training dan ditambahkan setelah proses perkalian bobot.
- $W_{3,j,i}$: Weight (bobot) yang menghubungkan neuron ke- i pada hidden layer kedua dengan neuron output ke- j . Bobot ini menentukan besarnya pengaruh setiap neuron terhadap nilai output.
- $h_{2,i}$: Output (aktivasi) neuron ke- i pada hidden layer kedua setelah melewati fungsi aktivasi ReLU. Nilai ini menjadi masukan (input) bagi output layer.
- i : Indeks neuron pada hidden layer kedua, dengan rentang 1 sampai 64.
- j : Indeks neuron pada output layer, $0 = Forward$, $1 = Turn Left$, dan $2 = Turn Right$.

Q Formula

$$\begin{aligned} Q_0 &= b_{3,0} + W_{3,0,1}h_{2,1} + W_{3,0,2}h_{2,2} + \dots + W_{3,0,64} h_{2,64} \\ Q_1 &= b_{3,1} + W_{3,1,1}h_{2,1} + W_{3,1,2}h_{2,2} + \dots + W_{3,1,64} h_{2,64} \\ Q_2 &= b_{3,2} + W_{3,2,1}h_{2,1} + W_{3,2,2}h_{2,2} + \dots + W_{3,2,64} h_{2,64} \end{aligned}$$

Q [Forward]

i= 0: (0.0605)(5.7913) = 0.350199
i= 1: (-0.1962)(3.1983) = -0.627502
i= 3: (0.5092)(2.1844) = 1.112186
i= 4: (0.4821)(2.3027) = 1.110013
i= 5: (0.6028)(2.7401) = 1.651617
i= 6: (0.6377)(2.4483) = 1.561332
... and 25 more terms

$$Q_0 = \text{bias} + \text{sum of weight} = 0.278427 + 20.799030 = \mathbf{13.37}$$

Q [Turn Left]

i= 0: (0.3952)(5.7913) = 2.288486
i= 1: (0.2699)(3.1983) = 0.863293
i= 3: (0.2007)(2.1844) = 0.438385
i= 4: (0.0649)(2.3027) = 0.149454
i= 5: (-0.0748)(2.7401) = -0.204849
i= 6: (0.1083)(2.4483) = 0.265239
... and 25 more terms

$$Q_1 = 0.278427 + 20.799030 \approx \mathbf{21.08}$$

Q [Turn Right]

i= 0: (0.6873)(5.7913) = 3.980324
i= 1: (0.3971)(3.1983) = 1.269960
i= 3: (-0.1393)(2.1844) = -0.304280
i= 4: (0.2064)(2.3027) = 0.475179
i= 5: (0.3206)(2.7401) = 0.878368
i= 6: (0.1698)(2.4483) = 0.415668
... and 25 more terms

$$Q_2 = 0.159266 + 20.905939 \approx \mathbf{21.07}$$

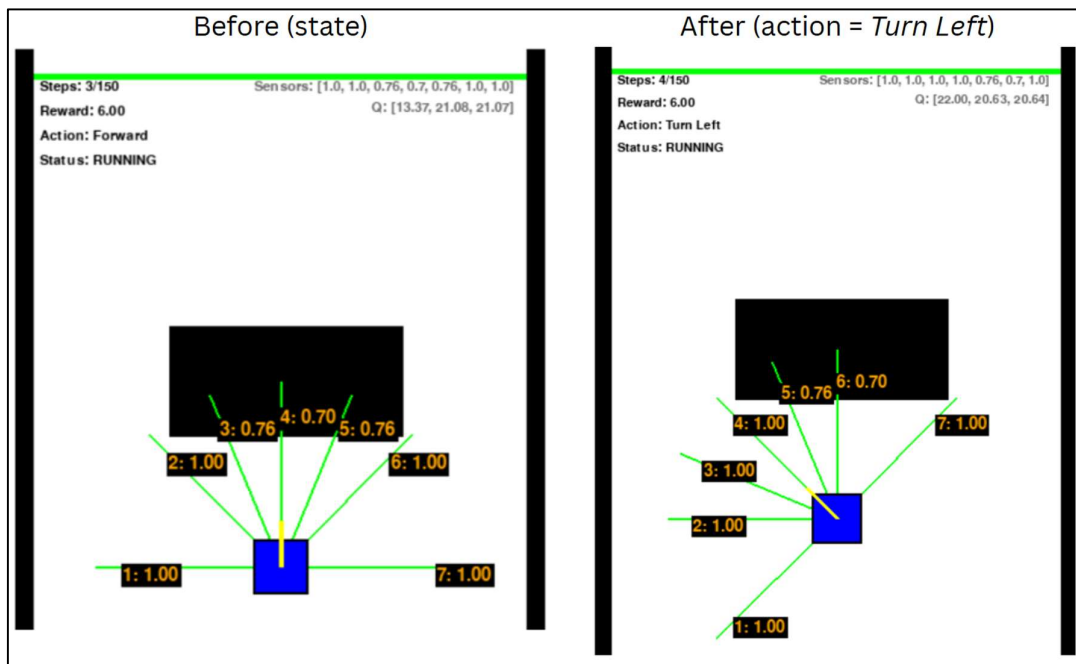
F) Hasil Akhir dan Keputusan Aksi

Tabel dibawah menunjukkan Kesimpulan *input* -> *output*

Input	Value	Normalized
Angle	0°	3.00
Sensor 1	1.00	1.00
Sensor 2	1.00	1.00
Sensor 3	0.76	0.76
Sensor 4	0.70	0.70
Sensor 5	0.76	0.76
Sensor 6	1.00	1.00
Sensor 7	1.00	1.00
Prev Action	Forward (permisalan)	1.00

Output	Aksi	Q-Value
Q ₀	Forward	13.37
Q ₁	Turn Left	21.08 (nilai tertinggi)
Q ₂	Turn Right	21.07

Sehingga aksi yang dipilih oleh agen apabila *exploitation* dilakukan dan bukan *exploration*, yaitu **Turn Left**. Karena **Q₁ (turn left)** memiliki *Q-value* tertinggi sebesar **21.08**. Hasil tersebut sesuai praktik model simulasi nyata, sesuai gambar di bawah.



DQN SCENARIOS

(Code B: Q-Training)

(Code C: Backpropagation)

Skenario: Episode 1 Step 1

A) Inisialisasi *Q-Network* dan *Target Network (hard copy network)* dengan *weight* dan *bias* sama)

Weight dan *Bias* NN itu setup awalnya adalah random, dalam rentang dan pembagian tertentu.

B) Epsilon (ϵ -greedy)

- *Exploration*: agen mengambil aksi random
- *Exploitation*: agen mengambil aksi berdasar *Q-value* tertinggi.

Contoh:

- Epsilon = 1 : action 100% random (*exploration*)
- Epsilon = 0,5 : action 50% random (*exploration*) dan 50% berdasar *Q-values* (*exploitation*)
- Epsilon = 0,1 : action 10% random (*exploration*) dan 90% berdasar *Q-values* (*exploitation*)

$$a = \begin{cases} \text{random action (exploration),} & \text{jika } r < \epsilon \\ \max_a Q(s, a) \text{ (exploitation),} & \text{jika } r \geq \epsilon \end{cases}$$

r : generasi angka acak rentang 0.00-1.00

C) Epsilon decay

Epsilon umumnya *decaying* atau berkurang setiap satu episode selesai. Menggunakan persamaan berikut.

$$\epsilon_t = \epsilon_{min} + (\epsilon_0 - \epsilon_{min}) \times (\epsilon_{decay})^t$$

- ϵ_t : Nilai epsilon pada episode ke- t .
- ϵ_0 : Nilai epsilon awal sebelum proses training dimulai.
- ϵ_{min} : Nilai epsilon minimum yang menjadi batas terendah selama proses decay.
- ϵ_{decay} : Faktor epsilon decay yang mengurangi nilai epsilon secara bertahap.
- t : Indeks atau nomor episode selama proses training.

D) Contoh Perhitungan Epsilon Decay

Diketahui:

- $\epsilon_0 = 1,00$
- $\epsilon_{min} = 0,01$
- $\epsilon_{decay} = 0,995$
- Episode ke- $t = 100$

Jawab:

$$\begin{aligned} \epsilon_t &= \epsilon_{min} + (\epsilon_0 - \epsilon_{min}) \times (\epsilon_{decay})^t \\ \epsilon_{100} &= 0,01 + (1,00 - 0,01) \times (0,995)^{100} \\ &= 0,01 + 0,99 \times 0,605770 \\ &= 0,01 + 0,599712 \\ &= 0,609712 \\ \epsilon_{100} &\approx 0,61 \end{aligned}$$

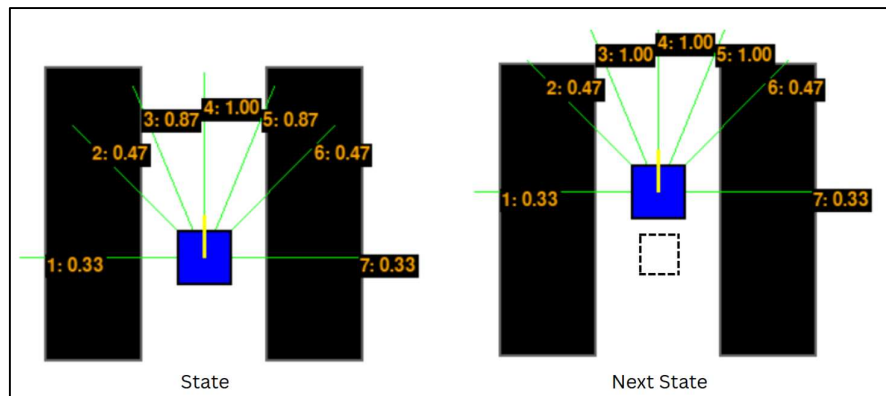
Jadi, pada episode ke-100, nilai epsilon menjadi sekitar **0.61**. Artinya, agen masih memiliki peluang sekitar **61%** untuk melakukan *exploration* (memilih aksi secara acak) dan sekitar **39%** untuk melakukan *eksploration* (memilih aksi berdasarkan *Q-value* tertinggi).

Episode	Current ϵ	Next ϵ
1	1	$0,01+(1,00-0,01) \times (0.995)^1 = \mathbf{0,995050}$
2	0,995050	$0,01+(1,00-0,01) \times (0.995)^2 = \mathbf{0,990125}$
3	0,990125	$0,01+(1,00-0,01) \times (0.995)^3 = \mathbf{0,985224}$
4	0,985224	$0,01+(1,00-0,01) \times (0.995)^4 = \mathbf{0,980348}$
5	0,980348	$0,01+(1,00-0,01) \times (0.995)^5 = \mathbf{0,975496}$

E) Skenario

Pada kasus ini, skenario berjalan pada episode 1 dan step 1. Artinya model masih belum terlatih, dan epsilon bernilai 1. Mengartikan agen akan selalu mengambil aksi acak atau *random*, tanpa mempedulikan nilai *Q-values* yang diperoleh melalui proses *feedforward*.

Step 1: Action (a) = exploration = 0 (forward)



State (Step 1):

Input	Value	Normalized
Angle	0°	3,00
Sensor 1	0,33	0,33
Sensor 2	0,47	0,47
Sensor 3	0,87	0,87
Sensor 4	1,00	1,00
Sensor 5	0,87	0,87
Sensor 6	0,47	0,47
Sensor 7	0,33	0,33
Prev Action	None	0,00

Next State (Step 2):

Input	Value	Normalized
Angle	0°	3,00
Sensor 1	0,33	0,33
Sensor 2	0,47	0,47
Sensor 3	1,00	1,00
Sensor 4	1,00	1,00
Sensor 5	1,00	1,00
Sensor 6	0,47	0,47
Sensor 7	0,33	0,33
Prev Action	Forward	1,00

F) Rewards

Reward/Penalty	Value	Keterangan
Clear Forward (CF)	2	Apabila sensor tengah kosong dan action adalah "Forward"
Success	50	Apabila agent mencapai <i>finish</i>
Turn Limit (TL)	-1	Apabila agent mencoba belok melebihi limit
Turn Repeat (TR)	-1	Apabila agent melakukan pola <i>turning</i> berulang (<i>ex: left -> right</i>)
Timeout	-25	Apabila <i>step</i> melebihi batas timeout (150 steps)
Collision	-25	Apabila agent melakukan tabrakan

Karena agen maju sat sensor tengah kosong, maka memenuhi reward **Clear Forward (CF)**

Step 1: reward (r) = 2

G) Replay Buffer

Replay buffer (D) menyimpan data transisi ($s_t, a_t, r_t, s_{t+1}, d_t$) yang diperoleh agen selama proses interaksi dengan lingkungan untuk digunakan kembali pada proses training.

$$D = (s_t, a_t, r_t, s_{t+1}, d_t)$$

Keterangan:

- s_t : State pada time step ke- t .
- a_t : Aksi yang dipilih pada time step ke- t .
- r_t : Reward yang diterima setelah melakukan aksi.
- s_{t+1} : State berikutnya setelah aksi dilakukan.
- d_t : Nilai done, 1 (True) jika episode berakhir dan 0 (False) jika episode berlanjut.

Berdasarkan hasil interaksi sebelumnya maka *replay buffer* (D) yang disimpan adalah sebagai berikut.

Stored Replay Buffer:

s_t	: [3,00, 0,33, 0,47, 0,87, 1,00, 0,87, 0,47, 0,33, 0,00] (<i>state</i>)
a_t	: 0 (<i>action pada state</i>)
r_t	: 2 (<i>reward pada state</i>)
s_{t+1}	: [3,00, 0,33, 0,47, 1,00, 1,00, 1,00, 0,47, 0,33, 1,00] (<i>next state</i>)
d_t	: 0 (episode belum berakhir karena tidak <i>finish</i> maupun <i>collision</i>)

Mengartikan ukuran *Replay Buffer* bertambah sebanyak +1, sehingga sekarang **$D(\text{size}) = 1$**

H) Check Replay Buffer Size $D(\text{size})$

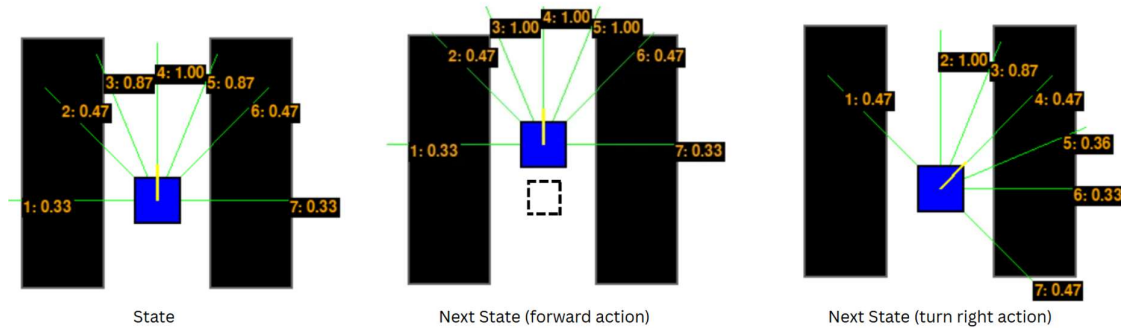
Pada simulasi ini **Batch Size** yang di set pada *hyperparameter* DQN adalah sebesar **64**. Mengartikan pelatihan akan dimulai apabila **$\text{Replay Buffer Size} \geq \text{Batch Size}$**

$D(\text{size}) \geq 64$: **Start Training**

$D(\text{size}) < 64$: **No Training**, continue gathering information

Dikarenakan pada step ini **$D(\text{size}) = 1$** maka **No Training**.

Skenario: Step 64 (Start Training) [CODE B: Q-TRAINING]



A) Check Replay Buffer Size $D(\text{size})$

Pada kasus ini $D(\text{size}) = 64$ sehingga **Start Training**.

B) Pengambilan Mini-Batch

Replay buffer menyimpan data transisi dalam bentuk

$$D = (s_t, a_t, r_t, s_{t+1}, d_t)$$

Pada setiap proses training, **64 sampel dipilih secara acak (mini-batch)** dari replay buffer.

Contoh salah satu data pada mini-batch:

s_t	: [3,00, 0,33, 0,47, 0,87, 1,00, 0,87, 0,47, 0,33, 0,00] (<i>state step 1</i>)
a_t	: 0 (<i>action forward pada step 1</i>)
r_t	: 2 (<i>reward step 1</i>)
s_{t+1}	: [3,00, 0,33, 0,47, 1,00, 1,00, 1,00, 0,47, 0,33, 1,00] (<i>next state</i>)
d_t	: 0 (<i>episode belum berakhir karena tidak finish maupun collision</i>)

C) Target Q-Value

Target Q-value adalah nilai target yang dijadikan acuan untuk *update Q-network*.

$$y_i = r_t + (1 - d_t) \gamma \max Q(s_{t+1})$$

Keterangan :

- *Rewards*: r_t
- *Done*: d_i
- *Gamma*: γ
- $Q(s_{t+1})$: [0.2461, 0.1763, **0.2506**] (diperoleh melalui proses *feedforward* menggunakan data s_{t+1})

Kalkulasi asli (episode belum selesai):

Diketahui : $r_t = 0$; $d_i = 0$; $\gamma = 0,99$; $\max Q = \mathbf{0.2506}$

Jawab : $y_i = 0 + (1 - 0) \cdot 0,99 \cdot \mathbf{0.2506} = \mathbf{2.2481}$

Kalkulasi percobaan (sekedar contoh kalkulasi apabila episode selesai karena *collision*):

Diketahui : $r_t = -25$; $d_i = 1$

Jawab : $y_i = -25 + (1 - 1) \cdot 0,99 \cdot 1,20 = \mathbf{-25}$

D) TD Error

TD Error adalah selisih antara *Target Q-value* dan *predicted Q-value* dari *online network*. Selisihnya digunakan untuk menghitung *loss*. Diketahui: $y_i = 2,481$ dan $Q(s_t) = [-0,0037, -0,0216, -0,0573]$

Maka TD Error adalah:

$$\delta_i = y_i - Q(s_t, a_t) = 2,481 - (-0,0037) = 2,2518$$

E) Loss Function

Loss dihitung sebagai *mean squared error* (MSE) antara *Target Q-value* dan *Predicted Q-value*. Digunakan untuk dasar *backpropagation* dan monitoring MSE.

$$L = \frac{1}{B} \sum_i^B (y_i - y_{i-1})^2$$

Keterangan:

- $L(\theta)$: nilai *loss*
- B : *batch size* (misal 64)
- y_i : *target Q-value* sampel ke- i
- y_{i-1} : *online Q-value* sampel sebelum ke- i

Konsep Kerja *Loss Function*:

- Bandingkan *Target Q* dengan *Predicted Q*
- Hitung selisihnya lalu kuadratkan hasilnya
- Ratakan untuk seluruh 64 sampel dalam mini-batch

Sampel	TD Error (δ)	Error ²
1	2,2518	5,0706
2	2,2422	5,0273
3	2,0506	4,2048
4	0,0019	0,0000
5	0,0767	0,0059
⋮	⋮	⋮
64	2,0742	4,3024

Maka Mean Squared Error (MSE):

$$L = \frac{1}{64} \sum_i^{64} (y_i - y_{i-1})^2$$

$$L = \frac{5,0706 + 5,0273 + 4,2048 + 0,0000 + 0,0059 + \dots + 4,3024}{64}$$

$$L = 2,584152 \text{ (Loss Value)}$$

- **loss kecil** → *Q-Network* semakin dekat ke *Target Network*
- **loss besar** → *Q-Network* masih jauh berbeda dari *Target Network*, diperlukan adanya perulangan untuk *backpropagation*.

Loss function menggunakan *Mean Squared Error* (MSE) antara *Target Q-value* dan *Predicted Q-value* pada seluruh sampel *minibatch*. Nilai *loss* ini kemudian digunakan dalam *Backpropagation* untuk memperbarui *Q-Network*.

Skenario: Backpropagation setelah Q-training
[CODE C: BACKPROPAGATION]

1. Gradient Loss

$Q = \text{predicted Q-Value} = -0,0037$

$y = \text{Target Q-Value} = 2,2481$

$$L = (y - Q)^2$$

Turunan terhadap $L = (y - Q)^2$:

$$\frac{\partial L}{\partial Q} = 2(Q - y)$$

$$\frac{\partial L}{\partial Q} = 2(-0,0037 - 2,2481) = 2(-2,2518)$$

$$\frac{\partial L}{\partial Q} = -4.5036 \text{ (Gradient Loss Value)}$$

Nilai ini merupakan *gradient loss* terhadap output Q yang menunjukkan bahwa prediksi model lebih rendah dari target, sehingga proses pembelajaran akan mendorong peningkatan nilai Q.

2. Definisi Model

$$Q = \sum_j W_j^{(4)} h_j^{(3)} + b^{(4)}$$

- Q : Output dari NN, yaitu *predicted Q-value* pada DQN yang merepresentasikan estimasi nilai aksi pada suatu *state*.
- $W_j^{(4)}$: Weight yang menghubungkan neuron ke-j pada *hidden layer* ke *output*.
- $h_j^{(3)}$: Nilai aktivasi neuron ke-j pada *hidden layer* ke-3, yang merupakan hasil transformasi dari *layer* sebelumnya dan fungsi aktivasi ReLU.
- $b^{(4)}$: Bias pada *layer output*, yaitu konstanta tambahan yang memungkinkan model melakukan pergeseran (*shift*) terhadap *output*.

3. Gradient Weight

$$\frac{\partial L}{\partial W_j^{(4)}} = \frac{\partial L}{\partial Q} \cdot \frac{\partial Q}{\partial W_j^{(4)}}$$

Keterangan:

- $\frac{\partial L}{\partial W_j^{(4)}}$: Gradien loss terhadap *weight* menentukan besar dan arah *update*
- $\frac{\partial L}{\partial Q}$: Gradien loss, misal mengikuti nilai *gradient loss* yang sebelumnya, yaitu -4.5036
- $\frac{\partial Q}{\partial W_j^{(4)}}$: Pengaruh aktivasi *weight* terhadap *output* (misal 5,7913)

Maka kalkulasi *gradient weight*:

$$\frac{\partial L}{\partial W_j^{(4)}} = (-4.5036) \cdot 5,7913 = -26.081555 \text{ (Gradient Weight)}$$

4. Gradient Bias

Misal bias aktif pada layer akhir adalah **-0.2089**, maka kalkulasi *gradient bias*:

$$\frac{\partial L}{\partial b^{(4)}} = \frac{\partial L}{\partial Q} \cdot \frac{\partial Q}{\partial b^{(4)}}$$
$$\frac{\partial L}{\partial b^{(4)}} = (-4.5036) \cdot (-0.2089) = 0.940628 \text{ (Gradient Bias)}$$

5. Update Parameter Weight

$$W_j^{new} = W_j - \alpha \cdot \frac{\partial L}{\partial W_j^{(4)}}$$

Keterangan:

- W_j : *weight* ke-j
- α : *learning rate* (misal 0,001)
- $\frac{\partial L}{\partial W_j^{(4)}}$: *gradient weight*

Kalkulasi:

$$W_j^{new} = 0.5 - (0.001 \times -26.081555)$$
$$W_j^{new} = 0.526082$$

Makna *update weight*:

- Perubahan *weight* ditentukan oleh arah dan besar gradien
- Gradien negatif → *weight* meningkat (arah menuju target)
- Besar perubahan dikontrol oleh *learning rate* (α)
- Aktivasi neuron sebelumnya mempengaruhi kontribusi *update*
- *Update* bertujuan mengurangi *error* secara bertahap

6. Update Parameter Bias

$$b^{new} = b_j - \alpha \cdot \frac{\partial L}{\partial b^{(4)}}$$

Keterangan:

- b : parameter bias
- α : *learning rate* (misal 0,001)
- $\frac{\partial L}{\partial b^{(4)}}$: *gradient weight*

Kalkulasi:

$$b^{new} = 0.1 - (0.001 \times 0.940628)$$
$$b^{new} = 0.099059$$

Makna *update bias*:

- Bias berfungsi menggeser *output* secara keseluruhan
- Gradien negatif → bias meningkat untuk menaikkan *Q-value*
- *Update* bias tidak dipengaruhi aktivasi neuron (langsung dari error)
- *Learning rate* mengontrol seberapa besar pergeseran terjadi
- Membantu model menyesuaikan dasar prediksi *action*

Update Network

NILAI YANG BERUBAH

$$\theta = \{W^{(1)}, W^{(2)}, W^{(3)}, W^{(4)}, b^{(1)}, b^{(2)}, b^{(3)}, b^{(4)}\}$$

Yang di-update saat *backpropagation* adalah:

ONLINE (policy) network saja

Yang tidak di-update:

TARGET network TIDAK ikut *backprop*

TUJUAN UPDATE

$$Q_{online}(s, a) \rightarrow y$$

Action	Initial Q	Final Q	Change
Forward (Q ₀)	-0.0073	20.344	↑ 2.0417
Turn Left (Q ₁)	-0.0148	-0.1419	↓ 0.1271
Turn Right (Q ₂)	0.0208	0.0768	↑ 0.0560

- Q₀ berubah lebih tinggi menandakan *action* paling optimal
- Q₁ dan Q₂ juga ikut berubah (efek samping *update weight* dan *bias*)
- Q-values tidak langsung berubah drastis, namun secara perlahan.

UPDATE TARGET NETWORK

Update Target Network dilakukan Setiap 10 kali *training* terjadi, dengan cara hardcopy Q-network to Target Network.

RINGKASAN ALUR BACKPROPAGATION

1. Hitung *Gradient Loss*.
2. Hitung *Gradient Weight* dan *Bias*.
4. *Update Weight* dan *Bias* pada Q-Network.
5. *Update Target Network* secara berkala.